

Integrating with Databases

Overview

Learn to seamlessly connect your StudAI-generated applications to popular databases: Supabase, Firebase, Neon, and Upstash. This concise guide focuses on essential setup and core operations.

Supabase Integration

Quick Setup

1. **Create Project** on supabase.com with name and password.
2. **Install:** `npm install @supabase/supabase-js`
3. **Configure:**

```
import { createClient } from '@supabase/supabase-js';  
const supabase = createClient(URL, ANON_KEY);
```

4. **Environment:** Add `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` to your `.env`.

Core Operations

- **Auth:** Built-in signup/sign-in functions.
- **CRUD:**

```
// Insert  
supabase.from('tasks').insert({ title, user_id });  
// Read  
supabase.from('tasks').select('*').eq('user_id', userId);  
// Update  
supabase.from('tasks').update({ status }).eq('id', taskId);  
// Delete  
supabase.from('tasks').delete().eq('id', taskId);
```

- **Real-time:**

```
supabase.channel('tasks')  
  .on('postgres_changes', { table: 'tasks' }, payload => console.log(payload))  
  .subscribe();
```

Firestore

Setup

1. **Install:** `npm install firebase`
2. **Configure:** Obtain config from Firebase console.
3. **Initialize:**

```
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
const app = initializeApp(config);
const db = getFirestore(app);
```

Core Operations

- **Add:** `addDoc(collection(db, 'tasks'), { title, userId });`
- **Read:** `getDocs(collection(db, 'tasks'));`
- **Update:** `updateDoc(doc(db, 'tasks', id), { status });`
- **Real-time:** `onSnapshot(collection(db, 'tasks'), snapshot => ...);`

Neon PostgreSQL

Setup

1. **Install:** `npm install @neondatabase/serverless`
2. **Configure:**

```
import { neon } from '@neondatabase/serverless';
const sql = neon(process.env.DATABASE_URL);
```

Operations

- **Query:**

```
const tasks = await sql`SELECT * FROM tasks WHERE user_id = ${userId}`;
```

- **Insert:**

```
await sql`INSERT INTO tasks (title,user_id) VALUES (${title},${userId})`;
```

Upstash Redis

Setup

1. **Install:** `npm install @upstash/redis`

2. **Configure:**

```
import { Redis } from '@upstash/redis';  
const redis = new Redis({ url, token });
```

Caching

- **Get:** `const cached = await redis.get(key);`
- **Set with TTL:** `await redis.setex(key, ttl, JSON.stringify(data));`

Migration & Seeding

- **Migrate:** Use SQL scripts for schema changes.
- **Seed:** Simple scripts inserting sample data with your chosen client.

Best Practices

- **Env Security:** Never commit `.env` to VCS.
- **Validation:** Validate inputs before DB operations.
- **Pooling:** Use connection pooling for production.
- **Indexes:** Add indexes on frequently queried columns.

Page count optimized for quick reference.